

PROGRAM 4:

```
//Author: Francis Reuben R - RVUN25CSE408
//Section: B, Roll no: 51
#include <stdio.h> // header file - standard i/o
#include <math.h> // header file - math functions (not strictly needed for this code)
// GCD Computational Function
int gcd(int a, int b) {
    int temp; //temp variable
    while (b != 0) { //while loop to compute gcd
        temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

void main() { //method main
    // Fibonacci Series Calculation
    int n, a = 0, b = 1, nT;
    printf("Enter the number of terms: ");
    scanf("%d", &n); // User input
    printf("Fibonacci Series: ");
    // Loop to calculate and print the series
    for (int i = 0; i < n; i++) {
        if (i <= 1) // first or zeroeth term
            nT = i;
        else {
            nT = a + b;
            a = b;
            b = nT;
        }
        printf("%d ", nT);
    }
    // GCD Calculation
    printf("\nCOMPUTING GCD OF TWO NUMBERS");
    printf("\nENTER TWO NUMBERS : ");
    int num1, num2; // user input
    scanf("%d %d", &num1, &num2);
    if (num1 < 0) num1 = -num1;
    if (num2 < 0) num2 = -num2;
    if (num1 == 0 && num2 == 0)
        printf("GCD of 0 and 0 is undefined (or 0, depending on definition)\n"); // 0 conditions
    else
        printf("GCD of %d and %d is %d\n", num1, num2, gcd(num1, num2)); // gcd computed
}
```

PROGRAM 5:

```
//Author: Francis Reuben R - RVUN25CSE408
//Section: B, Roll no: 51
#include <stdio.h>//header file - standard i/o
#include <math.h>//header file - math functions
#include <string.h>//header file - string functions
int fact(int x) { //factorial computation iterative function
    if (x < 0)
        return 0;
    int result = 1; //handles zeroeth condition
    for (int i = 1; i <= x; i++)
        result *= i; return result; }

int nthFib(int n) { //nth Fibonacci computational iterative function
    if (n < 0) return 0;
    if (n <= 1) return n;
    int a = 0; // F(i-2)
    int b = 1; // F(i-1)
    int c;      // F(i)
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b; // 'a' takes the value of the previous 'b'
        b = c; // 'b' takes the new calculated term 'c'
    }
    return b; }

void main(){
    printf("PROGRAM TO FIND FACTORIAL OF A NUMBER.");
    printf("\nENTER A NUMBER :");
    int num; scanf("%d",&num); //user input
    printf("\nFACTORIAL OF %d = %d\n",num, fact(num)); //factorial
    printf("\nPROGRAM TO FIND nth TERM OF THE FIBONACCI SERIES.");
    printf("\nENTER A NUMBER :");
    scanf("%d",&num); num--; //user input: num is now the 0-based index
    char str[][3]={"st","nd","rd"}; //position word
    printf("\n%d%s TERM OF THE FIBONACCI SERIES is = %d\n",
        num,
        num >= 1 && num <= 3 ? str[num-1] : "th",
        nthFib(num)); //nth Fibonacci term
}
```

PROGRAM 6:

```
//Author: Francis Reuben R - RVUN25CSE408
//Section: B, Roll no: 51
#include <stdio.h>//header file - standard i/o
#include <math.h>//header file - math functions
#include <string.h>//header file - string functions#include <stdio.h>
int nthFib(int n){ //nth Fibonacci term computational recursive function
    return n <= 1 ? n : nthFib(n - 1) + nthFib(n - 2); }

int gcd(int a,int b){ //nth gcd term computational recursive function
    return b == 0 ? a : gcd(b, a % b); }

void main(){
    printf("PROGRAM TO FIND nth TERM OF THE FIBONACCI SERIES USING RECURSION SPECIFICALLY.");
    printf("\nENTER A NUMBER :"); int num; // user input
    scanf("%d",&num); num--;
    char str[][3]={"st","nd","rd"}; //position word
    printf("\n%d%s TERM OF THE FIBONACCI SERIES is = %d\n",num,num<=3?str[num-1]:"th",nthFib(num)); //nth Fibonacci term printed
    printf("\nPROGRAM TO COMPUTE GCD OF TWO NUMBERS USING RECURSION SPECIFICALLY.");
    printf("\nENTER TWO NUMBERS : "); int num1, num2; //user input
    scanf("%d %d",&num1, &num2);
    printf("GCD of %d and %d is %d\n", num1, num2, gcd(num1, num2)); //gcd printed
}
```

PROGRAM 7:

```
//Author: Francis Reuben R - RVUN25CSE408
//Section: B, Roll no: 51
#include <stdio.h>
#define MAX 100// macro
void insert(int A[], int *N, int num1, int index) { //insert function
    if (*N == MAX) { //checks if array is full
        printf("Array is full. Cannot insert.\n");
        return;
    }
    if (index < 0 || index > *N) { //checks if Index is within bounds
        printf("Invalid index.\n");
        return;
    }
    for (int i = *N; i > index; i--) { //inserts element
        A[i] = A[i - 1];
    }
    A[index] = num1; (*N)++;
    printf("Inserted %d at index %d.\n", num1, index);
}

void delete(int A[], int *N, int index) { //delete function
    if (*N == 0) { //checks if array is empty
        printf("Array is empty. Cannot delete.\n");
        return;
    }
    if (index < 0 || index >= *N) { //checks if Index is within bounds
        printf("Invalid index.\n");
        return;
    }
    for (int i = index; i < *N - 1; i++) //deletes element
        A[i] = A[i + 1];
    (*N)--;
    printf("Deleted element at index %d.\n", index);
}

void update(int A[], int N, int num1, int index) { //update function
    if (N == 0) { //checks if array is empty
        printf("Array is empty. Cannot update.\n");
        return;
    }
    if (index < 0 || index >= N) { //checks if Index is within bounds
        printf("Invalid index.\n");
        return;
    }
    A[index] = num1; //updates element
    printf("Updated index %d with value %d.\n", index, num1);
}

void display(int A[], int N) { //print function
    printf("Array elements: ");
    for (int i = 0; i < N; i++) {
        printf("%d ", A[i]);
    }
    printf("\n");
}
```

```
void main() { //main method
    int A[MAX], N = 0, choice, num1, index; //function variables
    while(1){ //infinite loop unless exited otherwise
        printf("\nArray Operations Menu \n");
        printf("1. Insert\n2. Delete\n3. Update\n4. Display\n5. Exit\nEnter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to insert: "); scanf("%d", &num1);
                printf("Enter index to insert at: "); scanf("%d", &index);
                insert(A, &N, num1, index);
                break;
            case 2:
                printf("Enter index to delete: "); scanf("%d", &index);
                delete(A, &N, index);
                break;
            case 3:
                printf("Enter new value: "); scanf("%d", &num1);
                printf("Enter index to update: "); scanf("%d", &index);
                update(A, N, num1, index);
                break;
            case 4:
                display(A, N);
                break;
            case 5:
                printf("Exiting program.\n");
                return;
            default:
                printf("Invalid choice. Try again.\n");
        }
    }
}
```

PROGRAM 8:

```
//AUTHOR: Francis Reuben R
// RVUN25CSE408 SEC B
#include <stdio.h>

// Function 1: Calculates the length of a string.
int stringLength(char str[]) {
    int length = 0;
    while (str[length] != '\0') {
        length++;
    }
    return length;
}

// Function 2: Counts substring matches (overlapping allowed).
int subStringMatching(char str1[], char str2[]) {
    int temp = 0;
    int flag = 0;
    int len1 = stringLength(str1);
    int len2 = stringLength(str2);

    for (int i = 0; i <= len1 - len2; i++) {
        flag = 1;

        for (int j = 0; str2[j] != '\0'; j++) {
            if (str1[i + j] != str2[j]) {
                flag = 0;
                break;
            }
        }

        if (flag == 1) {
            temp++;
        }
    }
    return temp;
}
```

```

// Function 3: Reverses the characters in a string in place.
void reverseString(char str[]) {
    int i, j;
    char temp;
    int len = stringLength(str);

    for (i = 0, j = len - 1; i < j; i++, j--) {
        // Swap str[i] and str[j]
        temp = str[i];
        str[i] = str[j];
        str[j] = temp;
    }
}

// Function 4: Copies one string (str1) to another (str2).
void stringCopy(char str1[], char str2[]) {
    int i;
    for (i = 0; str1[i] != '\0'; i++) {
        str2[i] = str1[i];
    }
    str2[i] = '\0'; // Null termination
}

// Function 5: Compares two strings lexicographically and prints the result.
void stringCompare(char str1[], char str2[]) {
    int i;

    // Loop until the end of either string or characters mismatch
    for (i = 0; str1[i] == str2[i] && str1[i] != '\0'; i++) {
        // Comparison logic is handled by the loop condition and final check
    }

    if (str1[i] < str2[i]) {
        printf("Result of comparison: '%s' is less than '%s'\n", str1, str2);
    }
    else if (str1[i] > str2[i]) {
        printf("Result of comparison: '%s' is less than '%s'\n", str2, str1);
    }
    else {
        printf("Result of comparison: '%s' is equals to '%s'\n", str1, str2);
    }
}

```

```

int main() {
    char s1[100] = "Programming in C";
    char s2[100] = "Programming";
    char s3[100] = "C";
    char s4[100] = "";
    char dest_copy[100];
    char reverse_test[100] = "ReverseMe";

    // Test stringLength
    printf("1. stringLength\n");
    printf("Length of \"%s\": %d\n", s1, stringLength(s1));
    printf("Length of \"%s\": %d\n", s3, stringLength(s3));
    printf("Length of empty string: %d\n", stringLength(s4));
    printf("\n");

    // Test subStringMatching
    char main_str[] = "abababa";
    char sub_str[] = "aba";
    printf("2. subStringMatching\n");
    printf("String: \"%s\", Substring: \"%s\"\n", main_str, sub_str);
    printf("Number of matches: %d\n", subStringMatching(main_str, sub_str));
    printf("\n");

    // Test reverseString
    printf("3. reverseString\n");
    printf("Original string: \"%s\"\n", reverse_test);
    reverseString(reverse_test);
    printf("Reversed string: \"%s\"\n", reverse_test);
    printf("\n");

    // Test stringCopy
    printf("4. stringCopy\n");
    stringCopy(s1, dest_copy);
    printf("Source: \"%s\", Destination: \"%s\"\n", s1, dest_copy);
    printf("\n");

    // Test stringCompare
    printf("5. stringCompare\n");
    stringCompare(s2, s1);
    stringCompare(s1, s2);
    stringCompare(s2, s2);
    stringCompare(s3, s1);

    return 0;
}

```



```
parallels@debian-gnu-linux-12:/media/psf/Home/Documents/f College Work/f Sem 1/f PC/lab$ gcc Prog8.c -o Prog8
parallels@debian-gnu-linux-12:/media/psf/Home/Documents/f College Work/f Sem 1/f PC/lab$ ./Prog8
1. stringLength
Length of "Programming in C": 16
Length of "C": 1
Length of empty string: 0

2. subStringMatching
String: "abababa", Substring: "aba"
Number of matches: 3

3. reverseString
Original string: "ReverseMe"
Reversed string: "eMesreveR"

4. stringCopy
Source: "Programming in C", Destination: "Programming in C"

5. stringCompare
Result of comparison: 'Programming' is less than 'Programming in C'
Result of comparison: 'Programming' is less than 'Programming in C'
Result of comparison: 'Programming' is equals to 'Programming'
Result of comparison: 'C' is less than 'Programming in C'
```